

Organizational

- Main focus is on Codexpert exercises and Quizzes
- For the notes; I will be mailing them to you at the beginning, afterwards there could be a website for them
- esaroc@ethz.ch with subject "GM-NumerCS ..."

FEEL FREE TO ASK ANY QUESTIONS!

Intro

For many problems we cannot compute the exact solution analytically so we compute an approximation.

This course is all about finding approximations that are accurate, computationally and memory efficient.

→ We are used to work with $x \in \mathbb{R}$, but a computer can store only a finite set of floating numbers.

Code 1.1.3: Rundungsfehler

```
1 a = 4/3; b = a - 1; c = 3*b; e = 1-c  
2 print(e) # exact should be 0, get 2.220446049250313e-16
```

Not a bug!

Rounding Error

Definition 1.1.1 Let $\tilde{x} \in \mathbb{R}^n$ be a approximation of $x \in \mathbb{R}^n$. The

The absolute error is $\|\tilde{x} - x\|$ and if $\|x\| \neq 0$ the relative error is $\|\tilde{x} - x\| / \|x\|$.

Notion of instability:

tiny error $\xrightarrow{\text{algorithm}}$ $\begin{cases} \text{tiny/moderate error} & \text{stable} \\ \text{large error} & \text{instable} \end{cases}$

Code Example 1 A formula can be correct but unstable

Now in this example we know $p(x) = (x-c)(x - \frac{1}{c})$

Expanding gives $p(x) = x^2 - (c + \frac{1}{c})x + 1$

⇒ use $\frac{-b \pm \sqrt{b^2 - 4ac}}{2}$ after some playing around we get:

$$x_{\pm} = \frac{(c + \frac{1}{c}) \pm (c - \frac{1}{c})}{2}$$

Especially $x_- = \frac{1}{c}$ and this is correct, but for large c :

$\frac{-b - \sqrt{D}}{2}$ we have $-b \approx c$ and $\sqrt{D} \approx c$, hence cancellation

Although errors have not really changed, but the relative error got bigger.

Extra digits $x = 0.123467*$ $y = 0.123456*$, * means later unreliable. Now subtract $x - y = 0.000011*$

⇒ by $0.000011*$ the error gets more important, so the relative error does get amplified.

- Underflow occurs when the magnitude of the result is smaller than the smallest positive number that the floating-point system can represent
- Overflow opposite of underflow.

DO NOT MIX ROUNDING ERROR WITH CANCELLATION

- Say computer cannot store x, y exactly but with rounding errors δ_1, δ_2 : $\hat{x} = x(1 + \delta_1)$, $\hat{y} = y(1 + \delta_2)$, $|\delta_i| \leq u$.
 - We then compute $z = x - y$ but actually $\hat{z} = f(\hat{x} - \hat{y})$, so $\hat{z} = x - y + x\delta_1 + y\delta_2$
 - Abs error is $|\hat{z} - z| \approx |x\delta_1 + y\delta_2|$ and relative err. is $\frac{|x\delta_1 + y\delta_2|}{|x - y|} \leq \frac{|x + y|}{|x - y|} u$, with u being machine precision
- errors get amplified by a factor!

Code expert: 1. Ableitung mit imag. Schritt

We want to approximate the derivative $f'(x)$, but suppose we cannot differentiate it analytically.

From Analysis we know:

Natural approx. is:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \Rightarrow f'(x) = \frac{f(x+h) - f(x)}{h}$$

Reminder: Taylor expansion

In general Taylor expansion of f around a point a is:

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!} (x-a)^k$$

with $a = x$ and evaluation at $x+h$

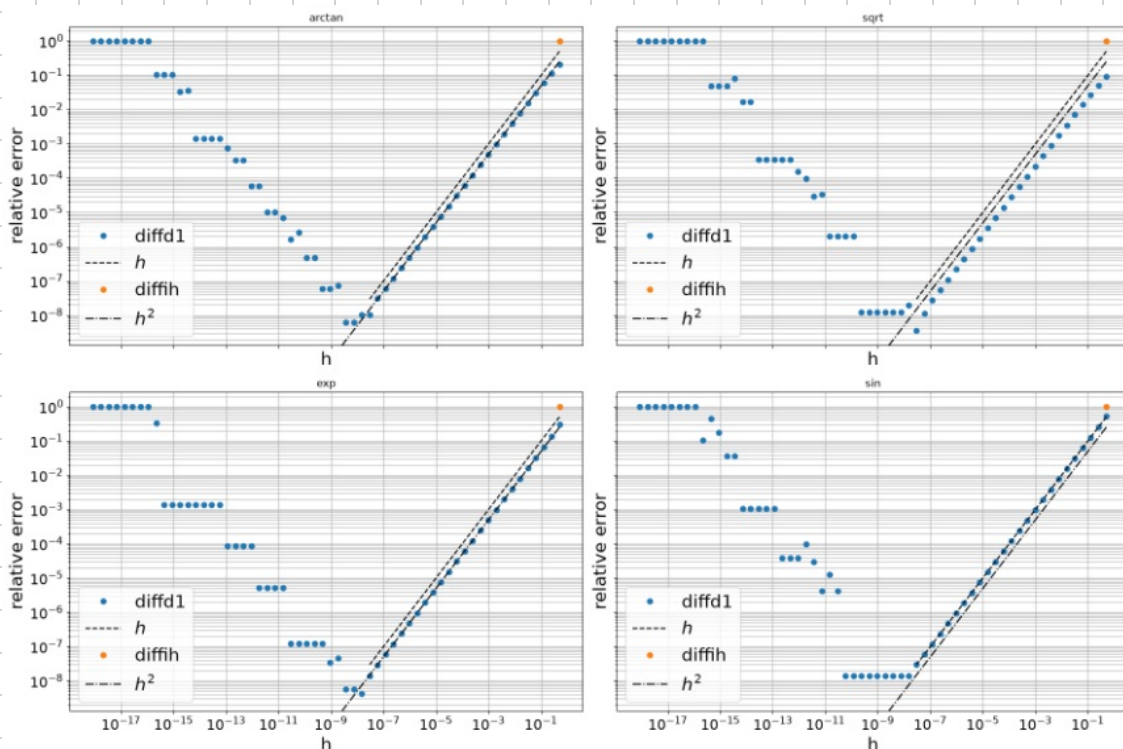
$$f(x+h) = f(x) + f'(x)h + \frac{1}{2} f''(x)h^2 + \frac{1}{6} f'''(x)h^3 + \dots$$

So using Taylor we get:

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \frac{1}{2} f''(x)h + \overset{\text{error}}{O(h^2)} \text{ hence error behaves}$$

like $O(h)$. (As $h \rightarrow 0$ h dominates h^2)

→ Then we make h as small as possible, but after a certain point relative error will explode because of cancellation as $f(x+h) \approx f(x)$.

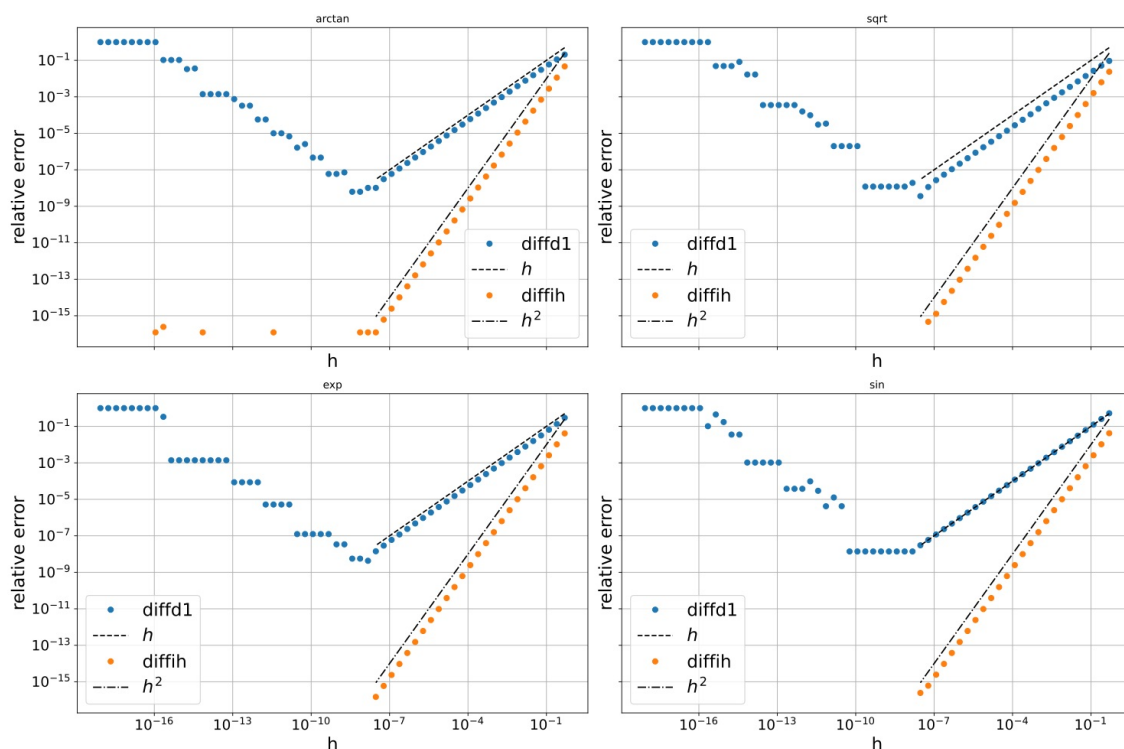


Imaginary Step

Now we use $f(x+ih) = f(x) + ih f'(x) - \frac{1}{2} h^2 f''(x) - \frac{i}{6} h^3 f'''(x) + \dots$

Then we only take imaginary part as $f'(x)$ belongs to it.

Therefore $f'(x) = \frac{\text{Im}(f(x+ih))}{h}$ now with error $O(h^2)$



each dot is a iteration with different h

→ What it lacks?

- it needs accurate evaluation of f , so not suitable for noisy data
- needs a very smooth/analytic function.

Coding Tips:

- you should use `np.imag(...)` somewhere 😊
- use the formulas in this note

Code expert: 1. Ableitung mit Richardson Beschleunigung

For the first TODO we want difference quotient of second order.

→ This is just a fancy way to get rid of the earlier dominating h in the term in $f'(x) + \text{error}$.

We do this by using Taylor expansion on $f(x-h)$:

$$\bullet f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + \frac{1}{6}f'''(x)h^3 + \dots$$

$$\bullet f(x-h) = f(x) - f'(x)h + \frac{1}{2}f''(x)h^2 - \frac{1}{6}f'''(x)h^3 + \dots$$

⇒ Subtract them:

$$f(x+h) - f(x-h) = 2f'(x)h + \frac{1}{3}f'''(x)h^3 + \dots \quad \rightarrow \text{even powers cancel}$$

⇒ Divide by $2h$

$$d(h) := \frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{1}{6}f'''(x)h^2 + \dots$$

Now the dominating term at the error is $O(h^2)$

What makes Richardson so special?

• We can achieve high accuracy without making h so small, so we can stop before cancellation becomes dominant

So it doesn't remove cancellation, as the name suggests it makes convergence faster.

→ The error has the structure:

$$f'(x) - d(h) = c_1 h^2 + c_2 h^4 + \dots + c_n h^{2n} + O(h^{2n+2}) \quad (A)$$

Turns out we can use different step sizes and combine them to make the leading term (e.g. h^2 above) disappear!

Example:

$$f'(x) - d(h/2) = c_1 \frac{1}{2^2} h^2 + c_2 \frac{1}{2^4} h^4 + \dots$$

Multiply this with 4 and subtract from (A) you can observe that $c_1 h^2$ terms cancel. This gives new approximation:

$$d_1(h) = \frac{4d(h/2) - d(h)}{3} \quad \text{and} \quad f'(x) - d_1(h) = O(h^4)$$

In general we define :

$R_{\ell,0} = d\left(\frac{h}{2^\ell}\right)$ for $\ell = 0, 1, 2, \dots$ then for $k = 1, 2, \dots, \ell$ the Richardson step is :

$$R_{\ell,k} = \frac{4^k R_{\ell,k-1} - R_{\ell-1,k-1}}{4^k - 1}$$

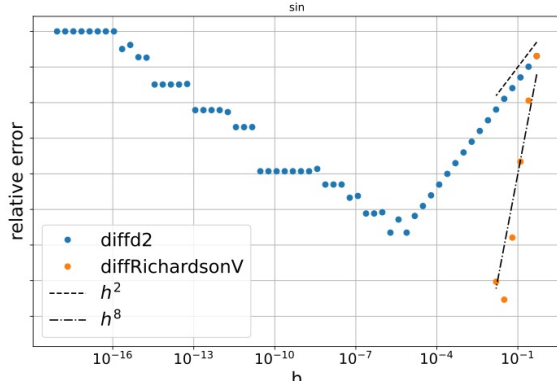
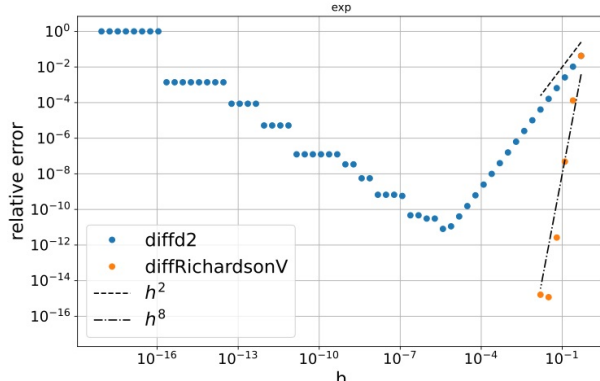
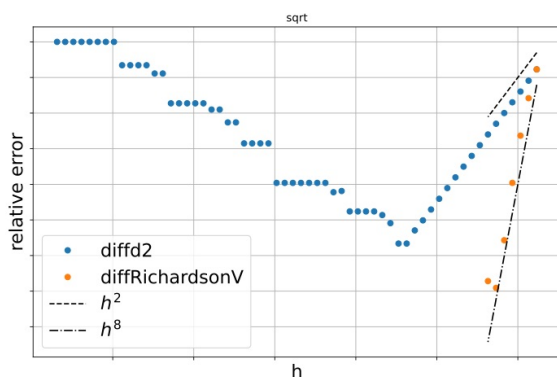
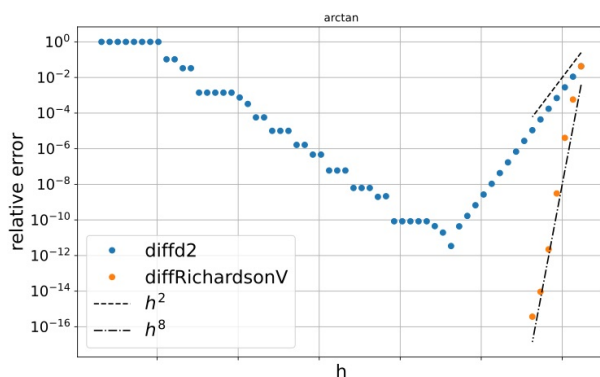
ℓ	k=0	k=1	k=2	k=3
0	$d(h) = R_{0,0}$			
1	$d(h/2) = R_{1,0} \rightarrow$	$R_{1,1}$		
2	$d(h/2^2) = R_{2,0} \rightarrow$	$R_{2,1} \rightarrow$	$R_{2,2}$	
3	$d(h/2^3) = R_{3,0} \rightarrow$	$R_{3,1} \rightarrow$	$R_{3,2} \rightarrow$	$R_{3,3}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

$\Rightarrow$ This can continue as long as f is smooth, as in each step we require higher derivatives
(1.1.8)

Error is then $f'(x) - R_{\ell,k} = C \cdot \left(\frac{h}{2^\ell}\right)^{2k+2}$

Cooling Tips:

- Take a look at Code 1.1.21
- errest is an estimate of the error because we do not know the real $f'(x)$
 $\Rightarrow \text{errest} = |R_{\ell,\ell} - R_{\ell-1,\ell-1}|$
- implement both methods first



Some Theory before next exercises

Computational cost is the number of elementary arithmetic operations

Well we are all familiar with asymptotic complexity, here we want to understand how the computational effort behaves as $n \rightarrow \infty$

↳ this is the part covering efficiency but it doesn't/can't give the actual picture about time.

↳ It is only about algorithmic scaling

Generic operations and their complexity

Operation	Math. form	Complexity
Dot product	$x^T y = \sum_{i=1}^n x_i y_i$	$\Theta(n)$
Outer product	$w v^T \in \mathbb{R}^{m \times n}$	$\Theta(mn)$
Matrix - vector	$y = Ax, A \in \mathbb{R}^{m \times n}$	$\Theta(mn)$
Matrix - Matrix	$C = AB, A \in \mathbb{R}^{m \times n}, B \in \mathbb{R}^{n \times p}$	$\Theta(mnp)$

Bemerkung 1.3.3 explains that in Matrix-vector product we just do a linear comb. of columns of A:

$$Ax = \sum_{j=1}^n x_j A_{:,j} \quad \text{sum}(x[j] * A[:, j] \text{ for } j \text{ in range}(A.\text{shape}[1]))$$

↳ Lets adapt it to Matrix - Matrix: $A \in \mathbb{R}^{m \times n}, B \in \mathbb{R}^{n \times p}$

Take one column of B, $b_i \in \mathbb{R}^n$ like x in Bemerkung 1.3.3

$$\Rightarrow Ab_i = \sum_{j=1}^n (b_i)_j * A_{:,j}$$

$$\Rightarrow \begin{bmatrix} | & | & & | \\ Ab_1 & Ab_2 & \dots & Ab_p \\ | & | & & | \end{bmatrix}$$

$$\Rightarrow AB = \sum_{j=1}^n \boxed{A_{:,j} B_{j,:}} \rightarrow \text{this is an outer product}$$

Bemerkung 1.3.4 $\text{rank}(AX) \leq \min \{ \text{rank}(A), \text{rank}(X) \}$ WHY?

Quick Info to remember

- Diagonal, upper and lower triangular matrices are closed under multiplication respectively.

$$\bullet \begin{bmatrix} d_1 & & 0 \\ & d_2 & \\ 0 & & d_n \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix} = \begin{bmatrix} d_1 a_{11} & d_1 a_{12} & \dots & d_1 a_{1m} \\ d_2 a_{21} & d_2 a_{22} & \dots & d_2 a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ d_n a_{n1} & d_n a_{n2} & \dots & d_n a_{nm} \end{bmatrix} = \begin{bmatrix} d_1 \mathbf{A}_{1,:} \\ \vdots \\ d_n \mathbf{A}_{n,:} \end{bmatrix}$$

$$\bullet \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{bmatrix} \begin{bmatrix} d_1 & & 0 \\ & d_2 & \\ 0 & & d_n \end{bmatrix} = \begin{bmatrix} d_1 a_{11} & d_2 a_{12} & \dots & d_m a_{1m} \\ d_1 a_{21} & d_2 a_{22} & \dots & d_m a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ d_1 a_{n1} & d_2 a_{n2} & \dots & d_m a_{nm} \end{bmatrix} = \begin{bmatrix} d_1 \mathbf{A}_{:,1} & \dots & d_m \mathbf{A}_{:,m} \end{bmatrix}$$

⇒ Left mult. with a diagonal matrix scales rows, right mult. with a diagonal matrix scales columns

⚠ You should understand that doing $\mathbf{D} @ \mathbf{A}$ in numpy treats both as $n \times n$ matrices. Hence complexity is $O(n^3)$. But isn't there a way we leverage the structure of the diagonal matrix?

→ This part is left for you to figure out.

Coding tips

- look at what `.diagonal()` does
- What happens to the shape of $\mathbf{x} \in \mathbb{R}^n$ in numpy when we do `x[:, np.newaxis]`?

⇒ Please look at the complexity of it afterwards!

- Broadcasting: $(n, 1) \times (n, m)$ and use `*` instead of `@`

Rank-1 matrix times a vector

Let $a, b, x \in \mathbb{R}^n$ and consider $y = (ab^T)x$.

→ This is the naive way, observe $C = ab^T \in \mathbb{R}^{n \times n}$. Remember outer product is in this case $O(n^2)$

We don't want a matrix!

$$\Rightarrow (ab^T)x = a(b^Tx)$$

$$\Rightarrow s = b^Tx \quad \dots \text{a scalar} \quad // \text{This is a dot product}$$

$$\Rightarrow y = as \quad // \text{This is just scaling}$$

→ This time we have only $O(n)$

Beispiel 1.3.19: Upper part of a low-rank matrix

Let $A, B \in \mathbb{R}^{n \times p}$ with $p \ll n$, $x \in \mathbb{R}^n$, and $\text{triu}(\cdot)$ does:

$$[\text{triu}(N)]_{ij} = \begin{cases} N_{ij} & , i \leq j \\ 0 & , i > j \end{cases}$$

Case 1: $p=1$. $A = a \in \mathbb{R}^n$ and $B = b \in \mathbb{R}^n$. Then $AB^T = ab^T$

for $n=4$:

$$1. \quad ab^T = \begin{bmatrix} a_1b_1 & a_1b_2 & a_1b_3 & a_1b_4 \\ a_2b_1 & a_2b_2 & a_2b_3 & a_2b_4 \\ a_3b_1 & a_3b_2 & a_3b_3 & a_3b_4 \\ a_4b_1 & a_4b_2 & a_4b_3 & a_4b_4 \end{bmatrix}$$

$$2. \quad y = \text{triu}(ab^T)x \quad \text{and let } R = \text{triu}(ab^T) \Rightarrow R = \begin{bmatrix} a_1b_1 & a_1b_2 & a_1b_3 & a_1b_4 \\ 0 & a_2b_2 & a_2b_3 & a_2b_4 \\ 0 & 0 & a_3b_3 & a_3b_4 \\ 0 & 0 & 0 & a_4b_4 \end{bmatrix}$$

3. Calculate $y = Rx$ the components are:

$$y_1 = a_1(b_1x_1 + b_2x_2 + b_3x_3 + b_4x_4)$$

$$y_2 = a_2(b_2x_2 + b_3x_3 + b_4x_4)$$

$$y_3 = a_3(b_3x_3 + b_4x_4)$$

$$y_4 = a_4b_4x_4$$

} Some terms reappear

$$4. \quad y_i = a_i \underbrace{\sum_{k=i}^n b_k x_k}_{s_i} \quad \dots \text{special structure is clear, they are cumulative sums from right}$$

⇒ So we should do $v = b \odot x \Rightarrow s_i = \sum_{k=1}^n v_k \Rightarrow y = a \odot s$

Case 2: general p

Use $AB^T = \sum_{l=1}^p A_{:,l} (B_{:,l})^T$ apply the previous $p=1$ idea to each term individually.

$$\Rightarrow (AB^T)_{ik} = \sum_{l=1}^p A_{il} B_{kl} \quad \text{and} \quad R = \text{triu}(AB^T) \\ \text{and} \quad y = Rx$$

⇒ $y_i = \sum_{k=1}^n (AB^T)_{ik} x_k$ then substitute $(AB^T)_{ik}$ to get

$$\Rightarrow y_i = \sum_{k=1}^n \left(\sum_{l=1}^p A_{il} B_{kl} \right) x_k = \sum_{l=1}^p A_{il} \left(\sum_{k=1}^n B_{kl} x_k \right)$$

Coding tips

- Use `np.cumsum()`
 - How could you use `x[::-1]` (reversing a list)?
 - ↳ Remember we said cumulative sum from the right!
- ⇒ Normal cumsum works like ...

Beispiel 1.3.22 Kronecker

We have $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{k \times l}$ their Kronecker is defined as:

$$A \otimes B = \begin{bmatrix} A_{11}B & \dots & A_{1n}B \\ \vdots & & \vdots \\ A_{m1}B & \dots & A_{mn}B \end{bmatrix} \in \mathbb{R}^{(mk) \times (nl)}$$

We want $y = (A \otimes B)x$ with $x \in \mathbb{R}^{nl}$ and $y \in \mathbb{R}^{mk}$ and we want to reuse our computations somehow.

- Naive approach complexity: $\Theta(m \cdot l \cdot n \cdot k)$

Observe we can split x into n blocks of vectors that are in $\mathbb{R}^l$

$$x = \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(n)} \end{bmatrix} \Rightarrow y = (A \otimes B)x \text{ let's do it for just one row:} \\ y_1 = A_{11} B x^{(1)} + A_{12} B x^{(2)} + \dots + A_{1n} B x^{(n)}$$

$$\text{So } y = \begin{bmatrix} \sum_{j=1}^n A_{1j} (Bx^{(j)}) \\ \vdots \\ \sum_{j=1}^n A_{mj} (Bx^{(j)}) \end{bmatrix}$$

↳ easy to spot that these get recomputed for each row and they are expensive

⇒ So we could calculate them for each j , so $Bx^{(j)}$ for all j , once and reuse them.

$$w^{(j)} = Bx^{(j)} \quad \text{and} \quad y^{(i)} = \sum_{j=1}^n A_{ij} w^{(j)}$$

$$\downarrow \\ \Theta(nkl)$$

$$\downarrow \\ \Theta(mnk)$$

$$\Rightarrow \Theta(nk(l+m))$$

Cooling Tips

- you might have to reshape x into a more easier way to work with, maybe think about the n blocks $x^{(j)} \in \mathbb{R}^l$:)
- at the end $y.\text{ravel}()$ is for flattening, this is needed as reshaping changes something

Code expert: Effiziente Konstruktion einer Vandermonde Matrix

Quick intro to interpolation

We have $p(x) = a_0 + a_1 x + a_2 x^2 + \dots + a_{n-1} x^{n-1}$... a poly of deg. $n-1$

Now we want to evaluate it at points $t_1, t_2, \dots, t_m$ for one point we would have $p(t_i) = a_0 + a_1 t_i + a_2 t_i^2 + \dots + a_{n-1} t_i^{n-1}$

$$\Rightarrow p(t_i) = \begin{bmatrix} 1 & t_i & t_i^2 & \dots & t_i^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

Do this simultaneously for all point $t_1, \dots, t_m$

$$\begin{bmatrix} p(t_1) \\ p(t_2) \\ \vdots \\ p(t_m) \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & t_1 & t_1^2 & \dots & t_1^{n-1} \\ 1 & t_2 & t_2^2 & \dots & t_2^{n-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & t_m & t_m^2 & \dots & t_m^{n-1} \end{bmatrix}}_{\text{Vandermonde matrix } V} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

Vandermonde matrix V

No need for tips here!

